Plano de implementação — AITrack + DataDrivr + Dirijabem

Baseline: commit `5593ec5` · **Data do plano:** 13/10/2026
Repositório canônico: `/home/pasteurjr/progreact/aitrack`
Backend ativo: `/home/pasteurjr/progreact/aitrack/aitrackdatadrivr`

1. Objetivo

Transformar a base atual em um sistema testável de monitoramento veicular com:

- simulador Tracker e simulador/replay Dirijabem;
- detecção comportamental baseada em tempo real;
- score de motorista que se recupera com o tempo;
- monitores executados continuamente no backend;
- alertas persistidos sem duplicação;
- ações de reconhecer, resolver e descartar;
- análise opcional com Claude, com fallback para regras;
- frontend sem dados mock;
- suporte a veículos Tracker e motoristas Dirijabem;
- testes automatizados e validação sem risco ao banco compartilhado.

2. Estado conhecido

Já existe

- `aitrackdatadrivr/simulator.py`: 10 veículos Tracker via TCP/9000.
- `dirijabem_simulator.py`: replay de rotas e viagens sintéticas.
- `dirijabem_continuous_simulator.py`: geração contínua no banco Dirijabem.
- API Flask na porta 5009 e frontend App 2 na porta 3003.
- Parsers Suntech, Queclink e Maxtrack.
- Eventos `harsh_accel`, `harsh_brake`, `sharp_turn` e `speeding`.
- Tabelas e CRUD de monitores, veículos monitorados, análises e alertas.
- Endpoints backend de `acknowledge`, `resolve` e `dismiss`.
- App 1 unificado na porta 3001.

Ainda não está pronto

- O `monitor_engine` não é iniciado pelo `run.py`.
- A análise atual é rule-based e não usa LLM.
- O score acumula penalidades e fica preso no piso 10.
- A detecção usa número de amostras, não `Δtempo`.
- Eventos em memória não têm limite nem sincronização.
- O frontend ainda usa mocks em alertas, eventos e detalhe de monitores.
- O motor comportamental ainda não ingere os motoristas Dirijabem.
- Não há suíte de testes do motor, monitores ou simuladores.
- Há credenciais e configurações duplicadas no código.

3. Regras de execução

1. Trabalhar somente em `aitrackdatadrivr/` e nos componentes do App 2; não editar as cópias legadas da raiz.
2. Criar um commit por tarefa, com o ID da tarefa na mensagem.
3. Rodar testes antes de cada commit e atualizar este documento com o resultado.
4. Nunca versionar `config/.env`, chaves, backups, `node_modules`, `build/` ou logs.
5. O banco `tracker` é o único banco permitido para escrita durante o desenvolvimento.
6. O banco `dirijabem` permanece somente leitura, exceto pelos simuladores já existentes.
7. Antes de qualquer migration destrutiva, gerar backup e testar primeiro em ambiente local/mock.
8. Nunca usar `TRUNCATE`, `DROP` ou `DELETE` amplo em banco compartilhado.

4. Fases e tarefas

Fase 0 — Base segura e testável

Objetivo: corrigir infraestrutura, dados comportamentais e testes antes de ligar os monitores.

ID	Entrega	Arquivos principais	Aceite
T0.1	Criar `.venv`, requirements consolidado e runner idempotente de migrations	`requirements.txt`, `migrations/apply.py`, `.gitignore`	venv instala; `--dry-run` funciona; migrations já aplicadas não são repetidas
T0.2	Centralizar Tracker/Dirijabem em `db_config.py`	`server/db_config.py`, `server/*`, simuladores	nenhuma senha em `.py`; iniciar fora do cwd funciona
T0.3	Corrigir imports e remover fallback falso de score 85	`server/monitor_api.py`	API retorna score real ou `null`, nunca valor inventado
T0.4	Limitar memória e adicionar lock	`server/behavioral_engine.py`	RSS controlado; consultas por veículo rápidas
T0.5	Detectar por `Δv/Δt` e `Δrumo/Δt`	`behavioral_engine.py`	testes cobrem aceleração, frenagem, curva, episódio de velocidade
T0.6	Score por janela deslizante	`behavioral_engine.py`	veículo sem evento recente retorna 100; perfis se diferenciam
T0.7	Persistência assíncrona de eventos	`behavioral_engine.py`, `db_handler.py`	fila estável; zero descartes no teste normal
T0.8	Consultas de eventos indexadas	migration `005`, `monitor_api.py`	endpoints de eventos respondem dentro dos limites definidos
T0.9	Tornar simulador Tracker realista	`simulator.py`	heading segue a rota; perfis geram diferenças reproduzíveis
T0.10	Testes unitários do motor	`aitrackdatadrivr/tests/`	testes passam sem escrever no banco
T0.11	Calibração por 30 minutos	simulador + API	good ≥85, moderate 55–85, poor <55; RSS <300 MB

Fase 1 — Monitores rule-based e ações reais

Objetivo: executar os monitores no mesmo processo do backend e gerar alertas reais.

ID	Entrega	Aceite
T1.1	Migration de cooldown e remoção controlada dos alertas mock	backup criado; nenhum alerta mock pendente
T1.2	Reescrever `analyze_monitor` lendo eventos do banco	análise registra janela, score, eventos e severidade reais
T1.3	Iniciar motor em thread daemon no `run.py`	ciclos executam sem bloquear API/socket
T1.4	Deduplicar alertas e permitir escalonamento	no máximo um alerta aberto por veículo/monitor/cooldown
T1.5	Adicionar `analyze now` e status do motor	endpoints retornam ciclo, próximas execuções e contadores
T1.6	Testar regras e execução de 30 minutos	análises persistem; good não gera alertas; poor gera alertas esperados

As ações de alerta devem persistir no banco:

```
pending -> acknowledged -> resolved
pending -> dismissed
acknowledged -> dismissed
```

Cada ação deve registrar data, operador e resultado. Falhas devem retornar erro HTTP explícito e não apenas `alert()` no navegador.

Fase 2 — Claude com fallback seguro

Objetivo: adicionar inteligência generativa sem tornar o sistema dependente da API externa.

ID	Entrega	Aceite
T2.1	Configurar `ANTHROPIC_API_KEY` apenas no `.env`	chave nunca aparece no Git/log
T2.2	Migration de configuração e metadados LLM	modelo, origem, tokens, tempo e erro persistidos
T2.3	Criar `server/llm_client.py`	resposta JSON validada; timeout/retry; erros capturados
T2.4	Integrar LLM após pré-filtro de regras	sem eventos novos não há chamada; falha usa regras
T2.5	Expor análises e métricas pela API	frontend recebe origem, resumo, padrões e recomendações
T2.6	Testar cliente falso e uma chamada real opcional	testes não dependem de rede; chamada real só com chave autorizada

O prompt deve exigir português, evidência dos dados recebidos, severidade, título, resumo, padrões, recomendações e confiança. O LLM não pode inventar localização, evento ou número.

Fase 3 — Frontend sem mock

Objetivo: fazer o App 2 mostrar somente dados do backend.

1. Criar tipos TypeScript para monitor, análise, alerta, evento e frota.
2. Substituir `mockAlerts` por `alertService.getAll/getStats`.
3. Ligar Reconhecer, Resolver e Descartar às APIs reais.
4. Mostrar análises e recomendações no detalhe do monitor.
5. Adicionar “Analisar agora”, ativar/desativar, criar e editar monitor.
6. Substituir eventos mock por `/api/events`.
7. Substituir catálogo mock por `/api/events/catalog`.
8. Remover `mockData/` e componentes mortos.
9. Validar as seis abas com Playwright ou teste manual documentado.

Aceite: `tsc`, build, ações e contagens do frontend coincidem com a API.

Fase 4 — Dirijabem dentro dos monitores

Objetivo: monitorar também motoristas do aplicativo Dirijabem.

1. Criar convenção `APP-<CODUSU>`.
2. Criar ingestão incremental de `dirijabem.localizacaodados` por watermark.
3. Permitir `veiculomonitor.tipo_veiculo = 'dirijabem'`.
4. Persistir eventos com `codusu`, sem exigir `veicod`.
5. Usar o `SCORE` da viagem mais recente como contexto adicional.
6. Testar `APP-1` em monitor ativo durante 15 minutos.

Não presumir unidades de `ACELLINATU` ou `VARDIRATU`; primeiro medir os dados e, se necessário, calcular por GPS/velocidade.

Fase 5 — Detectores e produção controlada

Implementar, com testes unitários e antirrepetição:

- idle excessivo;
- bateria baixa e corte de energia;
- horário incomum;
- reboque;
- colisão;
- condução agressiva;
- fadiga;
- geofence entry/exit;
- desvio de rota;
- manutenção;
- desperdício de combustível.

Manter `panic_button` desativado sem documentação do protocolo e `distracted_driving` planejado enquanto não houver sensor.

5. Roteiro de teste recomendado

5.1 Teste sem banco

Executar primeiro:

```
cd /home/pasteurjr/progreact/aitrack/aitrackdatadrivr
.venv/bin/python -m pytest -q tests
cd frontend
npx tsc --noEmit
npm test -- --watchAll=false --runInBand
npm run build
```

Esse estágio deve usar mocks de conexão e não alterar nenhum banco.

5.2 Teste Tracker local

1. Subir backend.
2. Subir `simulator.py` com 10 veículos.
3. Confirmar `/api/fleet/scores`, `/api/fleet/events`, `/api/fleet/stats`.
4. Conferir perfis e rotas.
5. Confirmar análises e alertas do monitor #4.

5.3 Teste Dirijabem

1. Usar primeiro `/api/dirijabem/user/<id>/start-synthetic`.
2. Confirmar viagem e pontos em `dirijabem`.
3. Só depois testar replay real.
4. Em seguida ativar ingestão `APP-<CODUSU>`.
5. Confirmar eventos e análise do motorista no monitor.

5.4 Teste de ações

Para um alerta de teste controlado:

```
curl -X PUT localhost:5009/api/alerts/ID/acknowledge \
  -H 'Content-Type: application/json' \
  -d '{"reconhecido_por":"teste"}'

curl -X PUT localhost:5009/api/alerts/ID/resolve

curl -X PUT localhost:5009/api/alerts/ID/dismiss
```

Conferir o status diretamente no banco `tracker`, sem apagar dados reais.

5.5 Teste de estabilidade

Rodar Tracker + Dirijabem por 30 minutos inicialmente e depois por 2 horas. Medir:

- RSS do backend;
- fila de eventos;
- eventos descartados;
- análises por monitor;
- alertas duplicados;
- latência dos endpoints;
- reinício sem corrupção.

6. Critérios de pronto

- Todas as tarefas T0–T5 registradas com commit e evidência.
- Testes Python e frontend verdes.
- Build sem warnings tratados como erro.
- Score diferencia perfis e se recupera.
- Eventos não crescem indefinidamente em memória.
- Motor de monitores inicia e sobrevive a exceções.
- Alertas não duplicam dentro do cooldown.
- Ações persistem corretamente.
- Claude funciona quando habilitado e não quebra o sistema quando indisponível.
- Frontend não contém referências a mocks.
- Tracker e Dirijabem aparecem no mesmo fluxo de monitoramento.
- App 1 continua funcionando na porta 3001.
- Nenhuma credencial foi adicionada ao Git.

7. Registro de execução

Data	Tarefa	Commit	Resultado
13/10/2026	Plano criado	—	Aguardando início da T0.1